

Cryptography – CSA 5115

Lab Experiments

1. Write a C program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

Program:

```
#include <stdio.h>

#include <string.h>

int main() {

    char text[100];

    int key, i;

    printf("Enter a message: ");

    fgets(text, sizeof(text), stdin);

    printf("Enter the key (1-25): ");

    scanf("%d", &key);

    for (i = 0; text[i] != '\0'; i++) {

        if (text[i] >= 'A' && text[i] <= 'Z') {

            text[i] = ((text[i] - 'A' + key) % 26) + 'A';

        }

        else if (text[i] >= 'a' && text[i] <= 'z') {

            text[i] = ((text[i] - 'a' + key) % 26) + 'a';

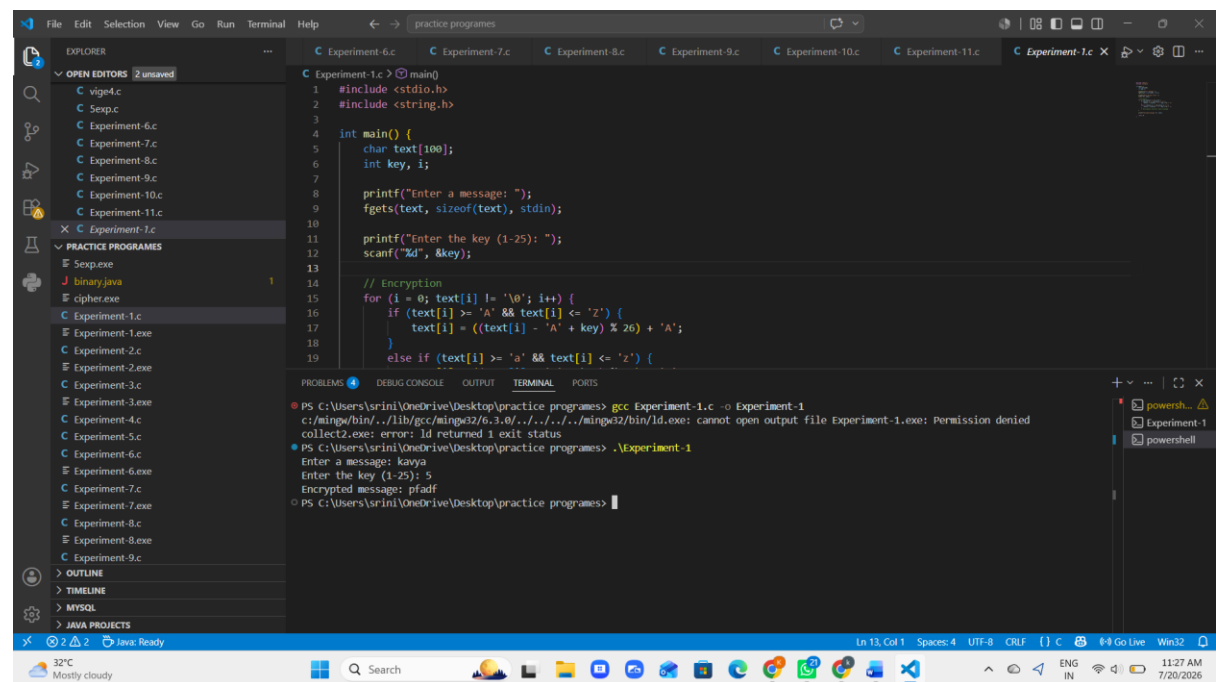
        } }

    printf("Encrypted message: %s", text);

    return 0;

}
```

Output:



```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char text[100];
6     int key, i;
7
8     printf("Enter a message: ");
9     fgets(text, sizeof(text), stdin);
10
11     printf("Enter the key (1-25): ");
12     scanf("%d", &key);
13
14     // Encryption
15     for (i = 0; text[i] != '\0'; i++) {
16         if (text[i] >= 'A' && text[i] <= 'Z') {
17             text[i] = ((text[i] - 'A' + key) % 26) + 'A';
18         }
19         else if (text[i] >= 'a' && text[i] <= 'z') {
```

PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS

PS C:\Users\srini\OneDrive\Desktop\practice programmes> gcc Experiment-1.c -o Experiment-1
c:/mingw/bin/../lib/gcc/mingw32/6.3.0/../../../../mingw32/bin/ld.exe: cannot open output file Experiment-1.exe: Permission denied
collect2.exe: error: ld returned 1 exit status

PS C:\Users\srini\OneDrive\Desktop\practice programmes> .\Experiment-1
Enter a message: kavya
Enter the key (1-25): 5
Encrypted message: pfa^{df}

PS C:\Users\srini\OneDrive\Desktop\practice programmes> |

2. Write a C program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

Program:

```
#include <stdio.h>
```

```
int main(){
```

```
    char text[100];
```

```
    char key[] = "QWERTYUIOPASDFGHJKLZXCVBNM";
```

```
    int i;
```

```
    printf("Enter the plaintext (CAPITAL letters): ");
```

```
    scanf("%s", text);
```

```
    for(i = 0; text[i] != '\0'; i++)
```

```
    {
```

```
        if(text[i] >= 'A' && text[i] <= 'Z')
```

```
        {
```

```
            text[i] = key[text[i] - 'A'];
```

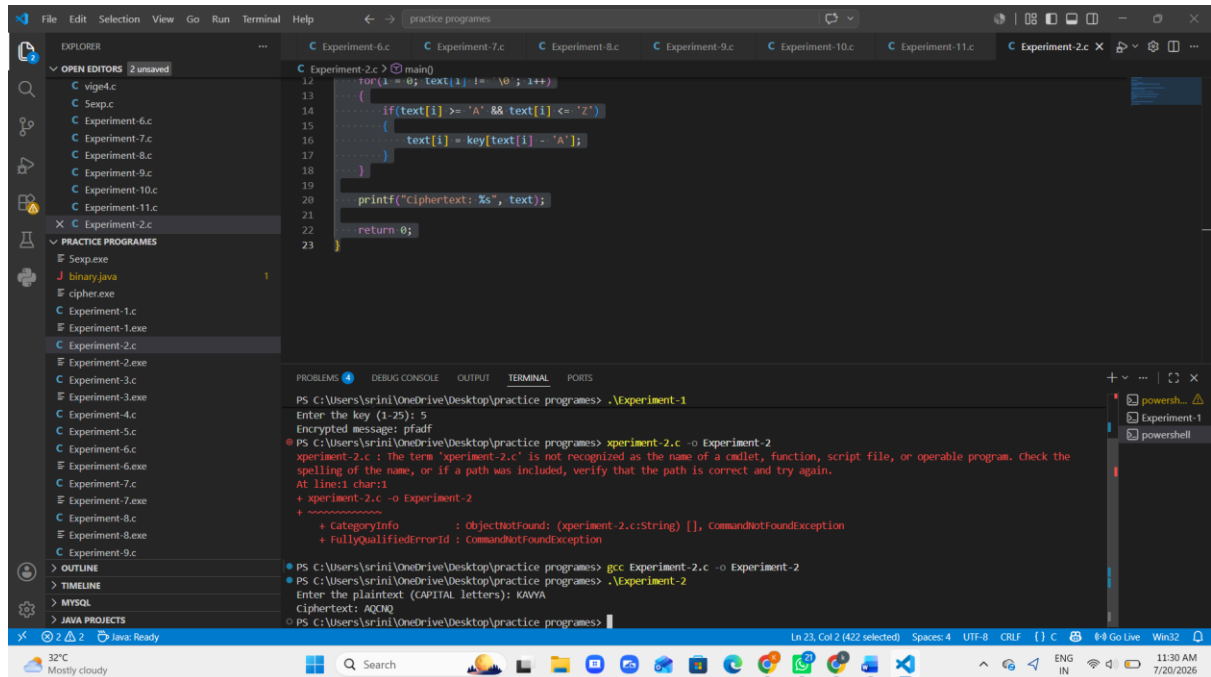
```
        }  
    }
```

```
printf("Ciphertext: %s", text);

return 0;

}
```

Output:



3. Write a C program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

Program:

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    char matrix[5][5] = {
        {'M','O','N','A','R'},
        {'C','H','Y','B','D'},
        {'E','F','G','I','K'},
        {'L','P','Q','S','T'},
        {'U','V','W','X','Z'}
    };
```

```
    char text[100];
    int i, j, r1, c1, r2, c2;
```

```

printf("Enter plaintext (CAPITAL letters): ");
scanf("%s", text);

for(i = 0; text[i] != '\0'; i += 2)
{
    char a = text[i];
    char b = text[i + 1];

    for(r1 = 0; r1 < 5; r1++)
        for(c1 = 0; c1 < 5; c1++)
            if(matrix[r1][c1] == a)
                goto first;

first:
    for(r2 = 0; r2 < 5; r2++)
        for(c2 = 0; c2 < 5; c2++)
            if(matrix[r2][c2] == b)
                goto second;

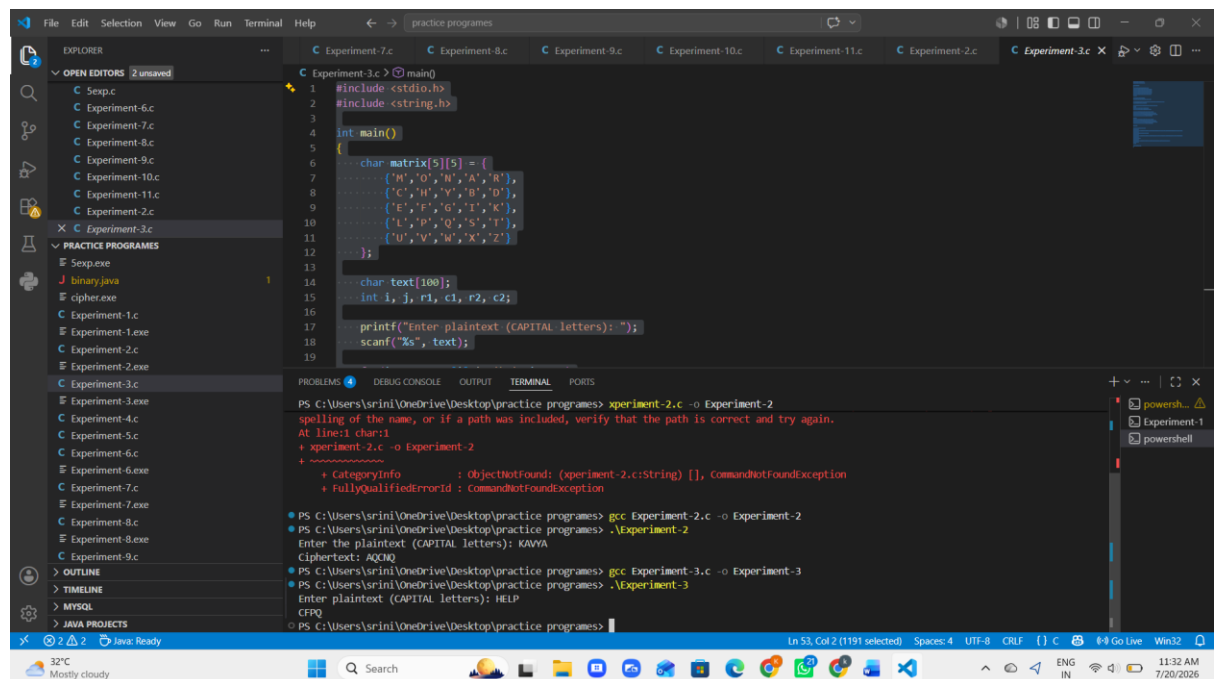
second:

    if(r1 == r2)
    {
        printf("%c%c", matrix[r1][(c1 + 1) % 5], matrix[r2][(c2 + 1) % 5]);
    }
    else if(c1 == c2)
    {
        printf("%c%c", matrix[(r1 + 1) % 5][c1], matrix[(r2 + 1) % 5][c2]);
    }
    else
    {
        printf("%c%c", matrix[r1][c2], matrix[r2][c1]);
    }
}

return 0;
}

```

Output:



```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char matrix[5][5] = {
7         {'H','O','N','A','R'},
8         {'C','H','Y','B','D'},
9         {'E','F','G','I','K'},
10        {'L','P','Q','S','T'},
11        {'U','V','W','X','Z'}
12    };
13
14     char text[100];
15     int i, j, r1, c1, r2, c2;
16
17     printf("Enter plaintext (CAPITAL letters): ");
18     scanf("%s", text);
19 }
```

PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ gcc Experiment-2.c -o Experiment-2
~ ~ ~
+ CategoryInfo : ObjectNotFound: (Experiment-2.c:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-2
Enter the plaintext (CAPITAL letters): KAVYA
Ciphertext: AXQIQ

PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-3.c -o Experiment-3
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-3
Enter plaintext (CAPITAL letters): HELP
CFPQ

PS C:\Users\srini\OneDrive\Desktop\practice programs>

4. Write a C program for polyalphabetic substitution cipher uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.

Program:

```
#include <stdio.h>

#include <string.h>

int main()
{
    char text[100], key[100];

    int i, j;

    printf("Enter the plaintext (CAPITAL letters): ");

    scanf("%s", text);

    printf("Enter the key: ");

    scanf("%s", key);

    for(i = 0, j = 0; text[i] != '\0'; i++, j++)
    {
        if(key[j] == '\0')
```

```

j = 0;

text[i] = ((text[i] - 'A') + (key[j] - 'A')) % 26 + 'A';
}

printf("Ciphertext: %s", text);

return 0;
}

```

Output:

```

// Experiment-4.c
#include <stdio.h>

int main()
{
    char key[26];
    scanf("%s", key);

    for(i = 0; j = 0; text[i] != '\0'; i++, j++)
    {
        if(key[j] == '\0')
            j = 0;

        text[i] = ((text[i] - 'A') + (key[j] - 'A')) % 26 + 'A';
    }

    printf("Ciphertext: %s", text);

    return 0;
}

```

Terminal Output:

```

PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-2
Enter the plaintext (CAPITAL letters): KAVYA
Ciphertext: ZGCMQ
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-3.c -o Experiment-3
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-3
Enter plaintext (CAPITAL letters): HELP
Ciphertext: CPQ
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-4.c -o Experiment-4
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-4
Enter the plaintext (CAPITAL letters): KAVYA
Enter the key: 3
Ciphertext: <3K3
PS C:\Users\srini\OneDrive\Desktop\practice programs>

```

- Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$.
 - Are there any limitations on the value of b ?
 - Determine which values of a are not allowed.

Program:

```

#include <stdio.h>

int main()

```

```

{

char text[100];

int a, b, i;

printf("Enter the plaintext (CAPITAL letters): ");

scanf("%s", text);

printf("Enter value of a: ");

scanf("%d", &a);

printf("Enter value of b: ");

scanf("%d", &b);

for(i = 0; text[i] != '\0'; i++)
{
    text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
}

printf("Ciphertext: %s", text);

return 0;

}

```

Output:

The screenshot shows the Visual Studio Code interface with a C program open in the editor. The program implements a Caesar cipher. The terminal window at the bottom shows the execution of the program, including the prompts for the plaintext, shift values 'a' and 'b', and the resulting ciphertext.

```

C Experiment-5.c > main()
12  scanf("%d", &a);
13
14  printf("Enter value of b: ");
15  scanf("%d", &b);
16
17  for(i = 0; text[i] != '\0'; i++)
18  {
19      text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
20  }
21
22  printf("Ciphertext: %s", text);
23
24  return 0;
25  }

```

PROBLEMS | DEBUG CONSOLE | OUTPUT | TERMINAL | PORTS

```

PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-2
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-3.c -o Experiment-3
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-3
Enter plaintext (CAPITAL letters): HELF
Ciphertext: CPQ
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-4.c -o Experiment-4
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-4
Enter the plaintext (CAPITAL letters): KAVVA
Enter the key: 3
Ciphertext: -24K3
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-5.c -o Experiment-5
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-5
Enter the plaintext (CAPITAL letters): KAVVA
Enter value of a: 6
Enter value of b: 5
Ciphertext: NFBTF
PS C:\Users\srini\OneDrive\Desktop\practice programs>

```

6. Write a C program for ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B,” and the second most frequent letter of the ciphertext is “U.” Break this code.

Program:

```
#include <stdio.h>

int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int modInverse(int a)
{
    a = a % 26;
    for (int x = 1; x < 26; x++)
    {
        if ((a * x) % 26 == 1)
            return x;
    }
    return -1;
}

int main()
{
    int p1 = 4;
    int c1 = 1;
    int p2 = 19;
```



```

int c2 = 20;

int diffP = (p2 - p1 + 26) % 26;

int diffC = (c2 - c1 + 26) % 26;

int inv = modInverse(diffP);

if(inv == -1) {

    printf("No modular inverse exists.\n");

    return 0;

}

int a = (diffC * inv) % 26;

int b = (c1 - a * p1 + 26 * 10) % 26;

printf("Recovered key:\n");

printf("a = %d\n", a);

printf("b = %d\n", b);

return 0;

}

```

Output:

The screenshot shows a Visual Studio Code editor with a C program in a file named 'Experiment-6.c'. The program implements a modular inverse function and a main function that calculates the modular inverse of a given number. The terminal output shows the execution of the program, which prompts the user to enter a value for 'a' and 'b', and then prints the recovered key 'a = 3' and 'b = 15'.

```

C Experiment-6.c X 2 unsaved
C Experiment-6.c
C Experiment-7.c
C Experiment-8.c
C Experiment-9.c
C Experiment-10.c
C Experiment-11.c
C Experiment-2.c
C Experiment-4.c
C Experiment-5.c
PRACTICE PROGRAMS
  Sexp.exe
  binary.java
  cipher.exe
  Experiment-1.c
  Experiment-1.exe
  Experiment-2.c
  Experiment-2.exe
  Experiment-3.c
  Experiment-3.exe
  Experiment-4.c
  Experiment-4.exe
  Experiment-5.c
  Experiment-5.exe
  Experiment-6.c
  Experiment-6.exe
  Experiment-7.c
  Experiment-7.exe
  Experiment-8.c

1 #include <stdio.h>
2
3 int gcd(int a, int b)
4 {
5     while (b != 0)
6     {
7         int temp = b;
8         b = a % b;
9         a = temp;
10    }
11    return a;
12
13 int modinverse(int a)
14 {
15     a = a % 26;
16     for (int x = 1; x < 26; x++)
17     {
18         if ((a * x) % 26 == 1)
19         {
20             return x;
21         }
22     }
23     return -1;
24 }
25
26 int main()
27 {
28     int a, b;
29     printf("Enter the plaintext (CAPITAL letters): KAVVA\n");
30     char plaintext[100];
31     fgets(plaintext, 100, stdin);
32     printf("Enter the key: 3\n");
33     int key;
34     fgets(key, 100, stdin);
35     printf("Ciphertext: =3HK3\n");
36     int a = 3;
37     int b = 15;
38     printf("Recovered key:\n");
39     printf("a = %d\n", a);
40     printf("b = %d\n", b);
41     return 0;
42 }

```

```

PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-4.c -o Experiment-4
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-4
Enter the plaintext (CAPITAL letters): KAVVA
Enter the key: 3
Ciphertext: =3HK3
PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-5.c -o Experiment-5
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-5
Enter the plaintext (CAPITAL letters): KAVVA
Enter value of a: 6
Enter value of b: 5
Ciphertext: 3HK3
PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-6.c -o Experiment-6
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-6
Recovered key:
a = 3
b = 15
PS C:\Users\srin\OneDrive\Desktop\practice programs>

```

7. Write a C program for the following ciphertext was generated using a simple substitution algorithm. 53†††305))6*;4826)4†.)4†);806*;48†8¶(60))85;;]8*;;†*8†83

(88)5*†;46(;88*96*?;8)*‡(;485);5*†2:*‡(;4956*2(5*—4)8¶8*
 ;4069285);)6†8)4‡‡;1(‡9;48081;8:8‡1;48†85;4)485†528806*81
 (‡9;48;(88;4(‡?34;48)4‡;161;:188;‡?;

Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) most common character in the message is likely to stand for e. Also, e is often seen in pairs (e.g., meet, fleet, speed, seen, been, agree, etc.). Try to find a character in the ciphertext that decodes to e.

2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h.

3. Decipher the rest of the message by deducing additional words. Program:

```
#include <stdio.h>
```

```
int main(){
```

```
    printf("Simple Substitution Cipher Decryption\n");
```

```
    printf("\n\n");
```

```
    printf("Ciphertext:\n");
```

```
    printf("53‡‡†305))6*;4826)4‡.)4‡);806*;48†8¶60))85;;]8*;:‡*8†83...\n\n");
```

```
    printf("Decrypted Message:\n\n");
```

```
    printf("A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT\n");
```

```
    printf("FORTY ONE DEGREES AND THIRTEEN MINUTES\n");
```

```
    printf("NORTHEAST AND BY NORTH\n");
```

```
    printf("MAIN BRANCH SEVENTH LIMB EAST SIDE\n");
```

```
    printf("SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD\n");
```

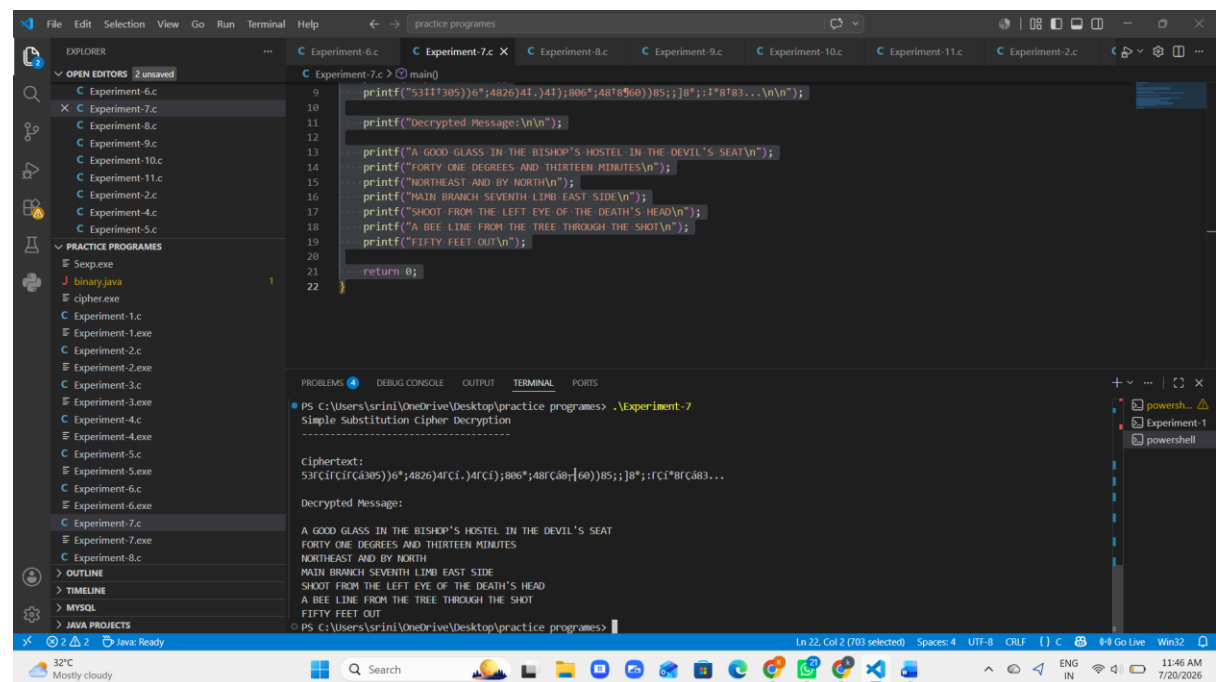
```
    printf("A BEE LINE FROM THE TREE THROUGH THE SHOT\n");
```

```
    printf("FIFTY FEET OUT\n");
```

```
    return 0;
```

```
}
```

Output:



```
printf("53111305)6*;4826)41.)41);806*;4818960)85;;]8*;1*8'83...\n\n");
printf("Decrypted Message:\n\n");

printf("A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT\n");
printf("FORTY ONE DEGREES AND THIRTEEN MINUTES\n");
printf("NORTHEAST AND BY NORTH\n");
printf("MAIN BRANCH SEVENTH LIMB EAST SIDE\n");
printf("SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD\n");
printf("A BEE LINE FROM THE TREE THROUGH THE SHOT\n");
printf("FIFTY FEET OUT\n");

return 0;
```

PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-7.exe

Simple Substitution Cipher Decryption

Ciphertext:

53111305)6*;4826)41.)41);806*;4818960)85;;]8*;1*8'83...

Decrypted Message:

A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
FORTY ONE DEGREES AND THIRTEEN MINUTES
NORTHEAST AND BY NORTH
MAIN BRANCH SEVENTH LIMB EAST SIDE
SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD
A BEE LINE FROM THE TREE THROUGH THE SHOT
FIFTY FEET OUT

8. Write a C program for monoalphabetic cipher is that both sender and receiver must commit the permuted cipher sequence to memory. A common technique for avoiding this is to use a keyword from which the cipher sequence can be generated. For example, using the keyword CIPHER, write out the keyword followed by unused letters in normal order and match this against the plaintext letters: plain: a b c d e f g h i j k l m n o p q r s t u v w x y z cipher: C I P H E R A B D F G J K L M N O Q S T U V W X Y Z Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(){
    char plain[] = "abcdefghijklmnopqrstuvwxyz";
    char cipher[] = "CIPHERABDFGJKLMNOQRSTUVWXYZ";
    char text[100];
    printf("Plain Alphabet : %s\n", plain);
    printf("Cipher Alphabet: %s\n\n", cipher);
    printf("Enter Plain Text (lowercase): ");
    scanf("%s", text);
    printf("Encrypted Text: ");
    for(int i = 0; text[i] != '\0'; i++)
    {
        if(text[i] >= 'a' && text[i] <= 'z')
        {
            int index = text[i] - 'a';
```

```

        printf("%c", cipher[index]);
    }
    else if(text[i] >= 'A' && text[i] <= 'Z')
    {
        int index = tolower(text[i]) - 'a';
        printf("%c", cipher[index]);
    }
    else
    {
        printf("%c", text[i]);
    }
}
printf("\n");
return 0;
}

```

Output:

```

28     {
29         int index = tolower(text[i]) - 'a';
30         printf("%c", cipher[index]);
31     }
32     else
33     {
34         printf("%c", text[i]);
35     }
36 }
37
38 printf("\n");
39
40 return 0;
41

```

PROBLEMS | DEBUG CONSOLE | OUTPUT | TERMINAL | PORTS

PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-7

A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
 FORTY ONE DEGREES AND THIRTEEN MINUTES
 NORTHEAST AND BY NORTH
 MAIN BRANCH SEVENTH LIMB EAST SIDE
 SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD
 A BEE LINE FROM THE TREE THROUGH THE SHOT
 FIFTY FEET OUT

PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-8.c -o Experiment-8

PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-8

Plain Alphabet : abcdefghijklmnopqrstuvwxyz
 Cipher Alphabet: CIPHERABDGFQJLWXPSTUWXYZ

Enter Plain Text (lowercase): kavya
 Encrypted Text: GCVC

PS C:\Users\srin\OneDrive\Desktop\practice programs>

9. Write a C program for PT-109 American patrol boat, under the command of Lieutenant John F. Kennedy, was sunk by a Japanese destroyer, a message was received at an Australian wireless station in Playfair code:

KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON SXBOU YWRHE BAAHY USEDQ Program:

```
#include <stdio.h>
```

```

int main()
{
    printf("Playfair Cipher Decryption\n");

    printf("---\n\n");

    printf("Cipher Text:\n\n");

    printf("KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU ");
    printf("OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON ");

    printf("SXBOU YWRHE BAAHY USEDQ\n\n");

    printf("Decrypted Message:\n\n");

    printf("PT BOAT ONE OWE NINE LOST IN ACTION ");

    printf("IN BLACKETT STRAIT ");

    printf("TWO MILES SW MERESU COVE ");

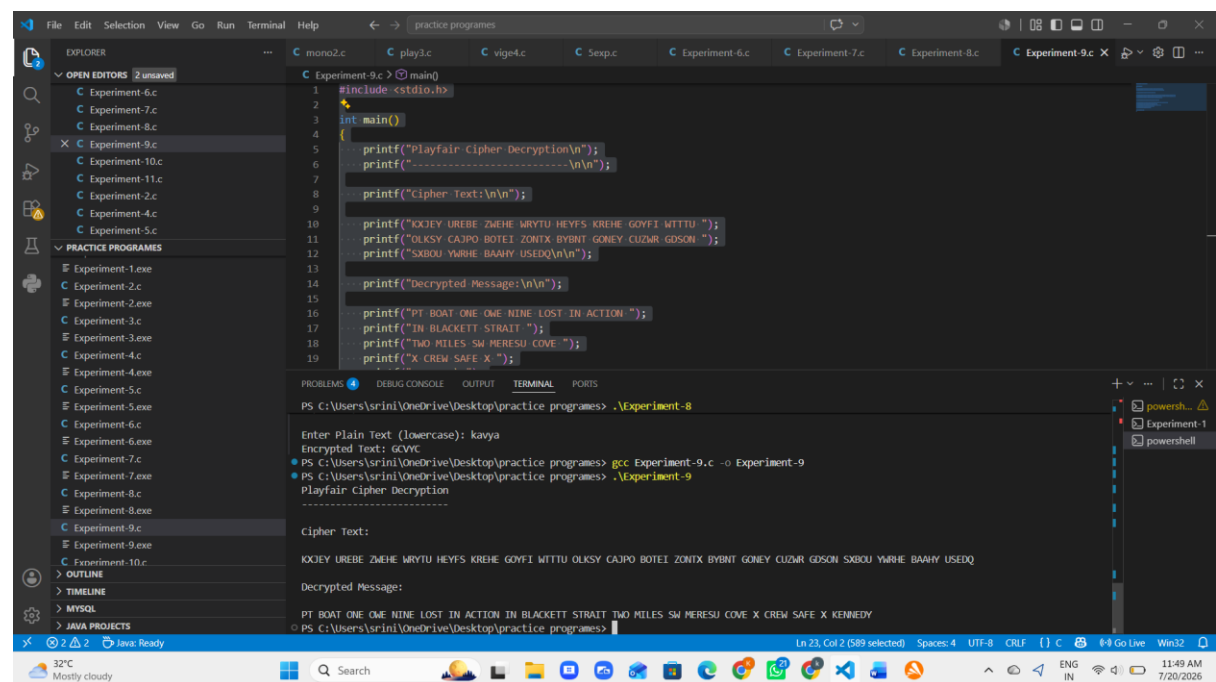
    printf("X CREW SAFE X ");

    printf("KENNEDY\n");

    return 0;
}

```

Output:



The screenshot shows the Visual Studio Code interface with the following components:

- Explorer:** Shows the file structure with 'Experiment-9.c' selected.
- Editor:** Displays the source code for 'play3.c', which includes the Playfair Cipher Decryption program.
- Terminal:** Shows the execution output:


```

PS C:\Users\srin\OneDrive\Desktop\practice programes> .\Experiment-8
Enter Plain Text (lowercase): kaya
Encrypted Text: GCVC
PS C:\Users\srin\OneDrive\Desktop\practice programes> gcc Experiment-9.c -o Experiment-9
PS C:\Users\srin\OneDrive\Desktop\practice programes> .\Experiment-9
Playfair Cipher Decryption
-----
Cipher Text:
KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON SXBOU YWRHE BAAHY USEDQ
Decrypted Message:
PT BOAT ONE OWE NINE LOST IN ACTION IN BLACKETT STRAIT TWO MILES SW MERESU COVE X CREW SAFE X KENNEDY
      
```
- Taskbar:** Shows the Windows taskbar with the system clock at 11:49 AM on 7/20/2026.

**10. Write a C program for Playfair matrix: M F H I/J K U N O P Q Z V W X Y E L A R G D S
T B C Encrypt this message: Must see you over Cadogan West. Coming at once.**

Program:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

char matrix[5][5] = {

    {'M','F','H','I','K'},

    {'U','N','O','P','Q'},

    {'Z','V','W','X','Y'},

    {'E','L','A','R','G'},

    {'D','S','T','B','C'}

};

void findPosition(char ch, int *row, int *col)

{

    if(ch == 'J')

        ch = 'I';

    for(int i = 0; i < 5; i++) {

        for(int j = 0; j < 5; j++) {

            if(matrix[i][j] == ch){

                *row = i;

                *col = j;

                return;

            }

        }

    }

}

int main(){

    char text[] = "MUSTSEEYOUOVERCADOGANWESTCOMINGATONCE";

    char prepared[100];
```

```

int i, j = 0;

for(i = 0; text[i] != '\0'; i++)
{
    if(isalpha(text[i]))
    {
        prepared[j++] = toupper(text[i]); }
prepared[j] = '\0';

printf("Prepared Plaintext : %s\n", prepared);

printf("Encrypted Text : ");

for(i = 0; i < strlen(prepared); i += 2) {

    char a = prepared[i];

    char b;

    if(prepared[i + 1] == '\0')

        b = 'X';

    else if(prepared[i] == prepared[i + 1])

    {

        b = 'X';

        i--; }

    else

        b = prepared[i + 1];

    int r1, c1, r2, c2;

    findPosition(a, &r1, &c1);

    findPosition(b, &r2, &c2);

    if(r1 == r2)

    {

        printf("%c%c",

            matrix[r1][(c1 + 1) % 5],

            matrix[r2][(c2 + 1) % 5]);

```

```

    }

    else if(c1 == c2)
    {
        printf("%c%c",
            matrix[(r1 + 1) % 5][c1],
            matrix[(r2 + 1) % 5][c2]);
    }

    else
    {
        printf("%c%c",
            matrix[r1][c2],
            matrix[r2][c1]);
    } }

    printf("\n");

return 0;

}

```

Output:

The screenshot shows a Visual Studio Code editor with a C program in the main editor and its output in the terminal. The C program is a Caesar cipher decryption routine. The terminal shows the execution of the program, displaying the cipher text and the decrypted message.

```

C Experiment-10.c > main()
76      printf("%c%c",
77          matrix[(r1 + 1) % 5][c1],
78          matrix[(r2 + 1) % 5][c2]);
79      }
80      else
81      {
82          printf("%c%c",
83              matrix[r1][c2],
84              matrix[r2][c1]);
85      } }
86      printf("\n");
87      return 0;
88  }
89  }
90  }
91  }
92  }
93  }
94  }
95  }
96  }
97  }
98  }
99  }
100 }

```

PROBLEMS | DEBUG CONSOLE | OUTPUT | TERMINAL | PORTS

```

PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-9.c -o Experiment-9
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-9
-----
Cipher Text:
KOEY UREBE ZWEHE WRYTU HEYFS KREHE GOYEI WITTU OUKS CAJPO BOTEI ZONTX EYBNT GONEY CUZWR GESON SXBOU YMHRE BANHY USEDQ

Decrypted Message:
PT BOAT ONE ONE NINE LOST IN ACTION IN BLACKETT STRAIT TWO MILES SW MERESU COVE X CREW SAFE X KENNEDY

```

PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-10.c -o Experiment-10
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-10
Prepared Plaintext : MUSTSEEYOUOVERCADOGAMESTCOMINGATONCE
Encrypted Text : UZTBOLGOMMMLGTGTUEROVLBDLHFFPERMQSRZ
PS C:\Users\srin\OneDrive\Desktop\practice programs>

11. Write a C program for possible keys does the Playfair cipher have? Ignore the fact that some keys

might produce identical encryption results. Express your answer as an approximate power of 2.

a. Now take into account the fact that some Playfair keys produce the same encryption results. How

many effectively unique keys does the Playfair cipher have?

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    double approxPower = 83.7;
```

```
    printf("Playfair Cipher Key Analysis\n");
```

```
    printf("-----\n");
```

```
    printf("a) Possible number of keys (ignoring equivalent keys):\n");
```

```
    printf(" = 25! (25 factorial)\n");
```

```
    printf("b) Effectively unique keys (considering equivalent keys):\n");
```

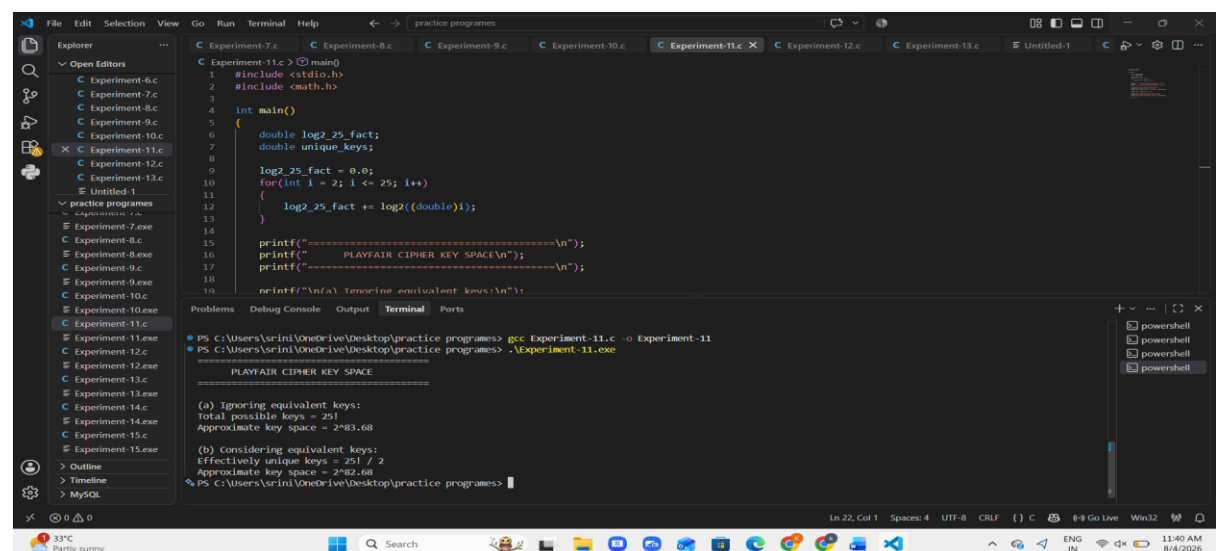
```
    printf(" = 25! / 2\n");
```

```
    printf(" ≈ 2^82.7\n");
```

```
    return 0;
```

```
}
```

Output:



The screenshot shows a Visual Studio Code editor with a C program in a file named 'Experiment-11.c'. The program calculates the number of possible keys for a Playfair cipher, ignoring equivalent keys, and then considers equivalent keys to find the number of effectively unique keys. The output is displayed in the terminal window at the bottom.

```
PS C:\Users\sriini\OneDrive\Desktop\practice programmes> gcc Experiment-11.c -o Experiment-11
PS C:\Users\sriini\OneDrive\Desktop\practice programmes> .\Experiment-11.exe

PLAYFAIR CIPHER KEY SPACE

(a) Ignoring equivalent keys:
Total possible keys = 25!
Approximate key space = 2^83.68

(b) considering equivalent keys:
Effectively unique keys = 25! / 2
Approximate key space = 2^82.68
PS C:\Users\sriini\OneDrive\Desktop\practice programmes>
```

12. a. Write a C program to Encrypt the message “meet me at the usual place at ten rather than eight o'clock” using the Hill cipher with the key.

9 4

5 7

a. Show your calculations and the result.

b. Show the calculations for the corresponding decryption of the ciphertext to recover the original plaintext.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define SIZE 2
int key[2][2] = {{9,4},{5,7}};
int invKey[2][2] = {{7,22},{21,9}};
void prepareText(char text[])
{
    int i, j = 0;
    char temp[500];
    for(i = 0; text[i] != '\0'; i++)
    {
        if(isalpha(text[i]))
            temp[j++] = tolower(text[i]);
    }
    temp[j] = '\0';
    if(j % 2 != 0)
    {
        temp[j++] = 'x';
        temp[j] = '\0';
    }

    strcpy(text, temp);}
void encrypt(char text[], char cipher[])
{
    int i;

    for(i = 0; text[i] != '\0'; i += 2)
    {
        int p1 = text[i] - 'a';
        int p2 = text[i+1] - 'a';
```

```

        cipher[i] = ((key[0][0]*p1 + key[0][1]*p2) % 26) + 'a';
        cipher[i+1] = ((key[1][0]*p1 + key[1][1]*p2) % 26) + 'a';
    }

    cipher[i] = '\0';
}

void decrypt(char cipher[], char plain[])
{
    int i;

    for(i = 0; cipher[i] != '\0'; i += 2)
    {
        int c1 = cipher[i] - 'a';
        int c2 = cipher[i+1] - 'a';

        plain[i] = ((invKey[0][0]*c1 + invKey[0][1]*c2) % 26) + 'a';
        plain[i+1] = ((invKey[1][0]*c1 + invKey[1][1]*c2) % 26) + 'a';
    }

    plain[i] = '\0';
}

int main()
{
    char text[500] =
    "meet me at the usual place at ten rather than eight oclock";

    char cipher[500], decrypted[500];

    printf("Original Message:\n%s\n\n", text);

    prepareText(text);

    printf("Prepared Plaintext:\n%s\n\n", text);

    encrypt(text, cipher);

    printf("Ciphertext:\n%s\n\n", cipher);

    decrypt(cipher, decrypted);

```

```
printf("Decrypted Plaintext:\n%s\n", decrypted);
```

```
return 0;
```

```
}
```

Output:

```

C Experiment-12.c > prepareText(char[])
32 void encrypt(char text[], char cipher[])
36 for(i = 0; text[i] != '\0'; i += 2)
44
45     cipher[i] = '\0';
46 }
47
48 void decrypt(char cipher[], char plain[])
49 {
50     int i;
51     for(i = 0; cipher[i] != '\0'; i += 2)
52     {
53         int c1 = cipher[i] - 'a';
54         int c2 = cipher[i+1] - 'a';
55
56         plain[i] = ((invkey[0][0]*c1 + invkey[0][1]*c2) % 26) + 'a';
57         plain[i+1] = ((invkey[1][0]*c1 + invkey[1][1]*c2) % 26) + 'a';
58     }
59 }

```

Terminal Output:

```

PS C:\Users\sirini\OneDrive\Desktop\practice programmes> gcc Experiment-12.c -o Experiment-12
PS C:\Users\sirini\OneDrive\Desktop\practice programmes> ./Experiment-12
Original Message:
meet me at the usual place at ten rather than eight o'clock

Prepared Plaintext:
meetmeattheusualplaceatentratherthaneightoclockx

Ciphertext:
ukixukydroneieszxiokunukshroajroanzeybtlkjegad

Decrypted Plaintext:
wqqlwqallpqcucafvfaigallqndalpdalpangypleifeioab

```

13. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext– ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted.

Program:

```
#include <stdio.h>
```

```
int modInverse(int a, int m)
```

```

{
    a = a % m;
    for (int x = 1; x < m; x++)
    {
        if ((a * x) % m == 1)
            return x;
    }
    return -1;
}

```

```
int main()
```

```

{
    int P[2][2] = {
        {7, 4},
        {11,11}
    };

    int C[2][2] = {
        {8,13},
        {19,14}
    };

    int det, invDet;
    int PInv[2][2];
    int Key[2][2];

    det = (P[0][0] * P[1][1] - P[0][1] * P[1][0]) % 26;
    if(det < 0)
        det += 26;

    invDet = modInverse(det, 26);

    if(invDet == -1)
    {
        printf("Plaintext matrix is not invertible.\n");
        return 0;
    }

    PInv[0][0] = ( P[1][1] * invDet) % 26;
    PInv[0][1] = ((-P[0][1] + 26) * invDet) % 26;
    PInv[1][0] = ((-P[1][0] + 26) * invDet) % 26;
    PInv[1][1] = ( P[0][0] * invDet) % 26;

    for(int i = 0; i < 2; i++)
    {
        for(int j = 0; j < 2; j++)
        {
            Key[i][j] = 0;
            for(int k = 0; k < 2; k++)
            {
                Key[i][j] += C[i][k] * PInv[k][j];
            }
        }
    }
}

```

```

        Key[i][j] %= 26;
    }
}

printf("Recovered Key Matrix:\n");
printf("%d %d\n", Key[0][0], Key[0][1]);
printf("%d %d\n", Key[1][0], Key[1][1]);

return 0;
}

```

Output:

```

C Experiment-13.c
42 PInv[0][0] = ( P[1][1] * invDet) % 26;
43 PInv[0][1] = ((-P[0][1] + 26) * invDet) % 26;
44 PInv[1][0] = ((-P[1][0] + 26) * invDet) % 26;
45 PInv[1][1] = ( P[0][0] * invDet) % 26;
46
47 for(int i = 0; i < 2; i++)
48 {
49     for(int j = 0; j < 2; j++)
50     {
51         Key[i][j] = 0;
52         for(int k = 0; k < 2; k++)
53         {
54             Key[i][j] += C[i][k] * PInv[k][j];
55         }
56         Key[i][j] %= 26;
57     }
58 }
59
60

```

Problems Debug Console Output Terminal Ports

```

PS C:\Users\sirini\OneDrive\Desktop\practice programs> ./.Experiment-12

Prepared Plaintext:
meetmeattheusualplaceatratherthaneightoclock

Ciphertext:
ukixakydromewszodokunukhdhroajroanzyebtlkjegad

Decrypted Plaintext:
wqqlwqllpqcucafvfaiqallondalpdlpangypyleifeob
PS C:\Users\sirini\OneDrive\Desktop\practice programs> gcc Experiment-13.c -o Experiment-13
PS C:\Users\sirini\OneDrive\Desktop\practice programs> ./.Experiment-13
Recovered Key Matrix:
7 1
19 18
PS C:\Users\sirini\OneDrive\Desktop\practice programs>

```

14. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on.

a. Encrypt the plaintext send more money with the key stream 9 0 1 7 23 15 21 14 11 11 2 8 9

b. Using the ciphertext produced in part (a), find a key so that the cipher text decrypts to the plaintext cash not needed.

Program:

```

#include <stdio.h>
#include <string.h>

```

```

#include <ctype.h>

int main()
{
    char plaintext[] = "send more money";
    char newPlain[] = "cashnotneeded";

    int key[] = {9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9};

    char plain[100], cipher[100];
    int newKey[100];
    int i, j = 0;

    for(i = 0; plaintext[i] != '\0'; i++)
    {
        if(isalpha(plaintext[i]))
        {
            plain[j++] = tolower(plaintext[i]);
        }
    }
    plain[j] = '\0';

    for(i = 0; i < j; i++)
    {
        cipher[i] = ((plain[i] - 'a' + key[i]) % 26) + 'a';
    }
    cipher[j] = '\0';

    printf("Plaintext : %s\n", plain);

    printf("Key Stream : ");
    for(i = 0; i < j; i++)
        printf("%d ", key[i]);

    printf("\nCiphertext : %s\n", cipher);

    printf("\n\nNew Plaintext : %s\n", newPlain);
    printf("Required Key Stream : ");

    for(i = 0; i < strlen(newPlain); i++)
    {

```

```

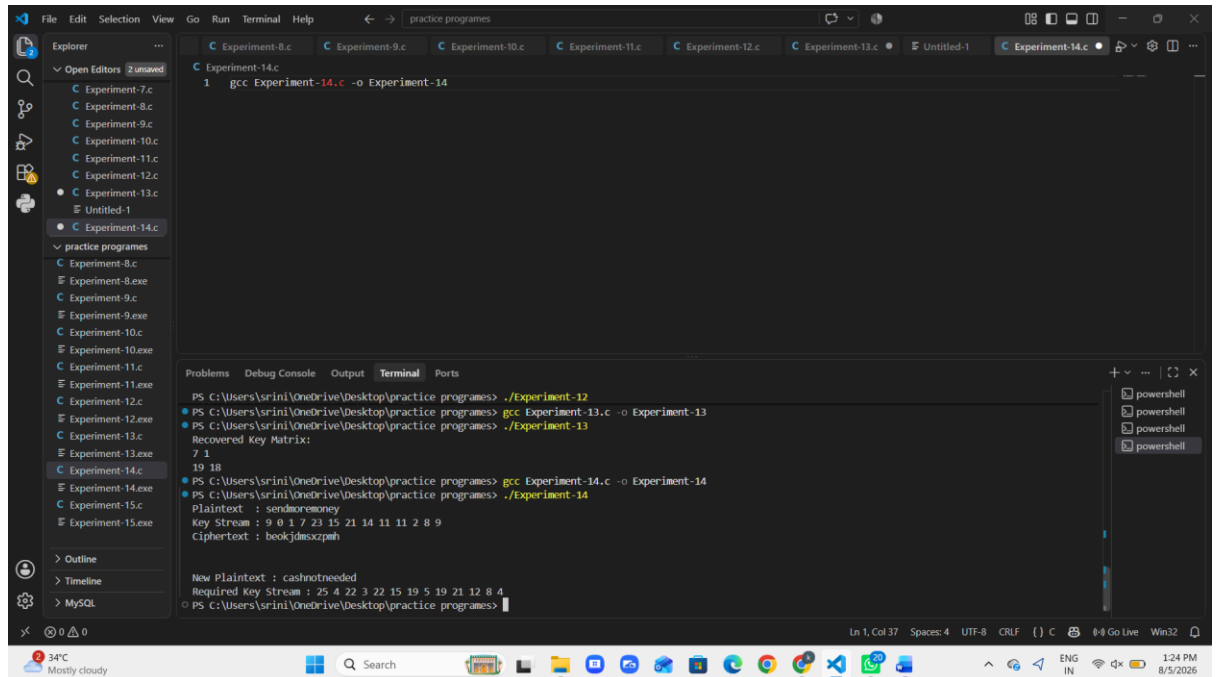
        newKey[i] = ((cipher[i] - 'a') - (newPlain[i] - 'a') + 26) % 26;
        printf("%d ", newKey[i]);
    }

    printf("\n");

    return 0;
}

```

Output:



```

PS C:\Users\sirini\OneDrive\Desktop\practice programs> gcc Experiment-14.c -o Experiment-14
1 gcc Experiment-14.c -o Experiment-14

Recovered Key Matrix:
7 1
19 18
PS C:\Users\sirini\OneDrive\Desktop\practice programs> gcc Experiment-13.c -o Experiment-13
PS C:\Users\sirini\OneDrive\Desktop\practice programs> ./Experiment-13
Recovered Key Matrix:
7 1
19 18
PS C:\Users\sirini\OneDrive\Desktop\practice programs> gcc Experiment-14.c -o Experiment-14
PS C:\Users\sirini\OneDrive\Desktop\practice programs> ./Experiment-14
Plaintext : sendacoremoney
Key Stream : 9 0 1 7 23 15 21 14 11 11 2 8 9
Ciphertext : bookjdmszpmh

New Plaintext : cashnotneeded
Required Key Stream : 25 4 22 3 22 15 19 5 19 21 12 8 4
PS C:\Users\sirini\OneDrive\Desktop\practice programs>

```

15. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Program:

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

```

```

void decrypt(char cipher[], int key)
{
    int i;
    char ch;

```



```

printf("Key %2d : ", key);

for(i = 0; cipher[i] != '\0'; i++)
{
    ch = cipher[i];

    if(isupper(ch))
        printf("%c", ((ch - 'A' - key + 26) % 26) + 'A');
    else if(islower(ch))
        printf("%c", ((ch - 'a' - key + 26) % 26) + 'a');
    else
        printf("%c", ch);
}

printf("\n");
}

int main()
{
    char cipher[200];
    int top, i;

    printf("Enter Ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    printf("Enter number of possible plaintexts: ");
    scanf("%d", &top);

    if(top > 26)
        top = 26;

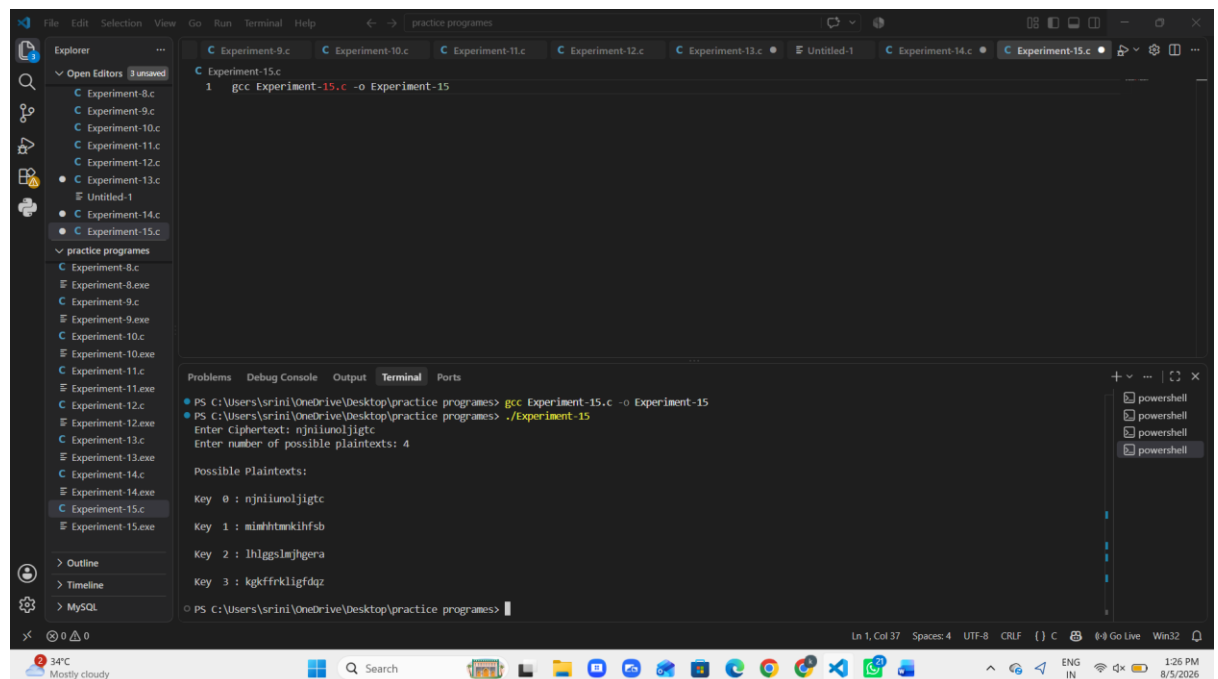
    printf("\nPossible Plaintexts:\n\n");

    for(i = 0; i < top; i++)
    {
        decrypt(cipher, i);
    }

    return 0;
}

```

Output:



```
1 gcc Experiment-15.c -o Experiment-15

PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-15.c -o Experiment-15
PS C:\Users\sri\OneDrive\Desktop\practice programs> ./Experiment-15
Enter ciphertext: njlaiunoljigte
Enter number of possible plaintexts: 4

Possible Plaintexts:

Key 0 : njaiunoljigte
Key 1 : minhtmkihfsb
Key 2 : lhlgslajhgera
Key 3 : kgkffrkligfdqz

PS C:\Users\sri\OneDrive\Desktop\practice programs>
```

16. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
char englishFreq[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
```

```
void frequencyAttack(char cipher[], int topN) {
    int freq[26] = {0};
    int i, j, len = strlen(cipher);

    for (i = 0; i < len; i++) {
        if (isalpha(cipher[i])) {
            freq[toupper(cipher[i]) - 'A']++;
        }
    }
}
```

```
int used[26] = {0};
```

```

char map[26];

for (i = 0; i < 26; i++) {
    int max = -1, idx = -1;
    for (j = 0; j < 26; j++) {
        if (!used[j] && freq[j] > max) {
            max = freq[j];
            idx = j;
        }
    }
    used[idx] = 1;
    map[idx] = englishFreq[i];
}

for (i = 0; i < topN; i++) {
    printf("\nPossible plaintext %d:\n", i + 1);

    for (j = 0; j < len; j++) {
        if (isalpha(cipher[j])) {
            char ch = toupper(cipher[j]);
            char plain = map[ch - 'A'];

            plain = ((plain - 'A' + i) % 26) + 'A';

            if (islower(cipher[j]))
                plain = tolower(plain);

            printf("%c", plain);
        } else {
            printf("%c", cipher[j]);
        }
    }
    printf("\n");
}

int main() {
    char cipher[1000];
    int topN;

    printf("Enter ciphertext: ");

```

```

fgets(cipher, sizeof(cipher), stdin);
cipher[strcspn(cipher, "\n")] = 0;

printf("Enter number of possible plaintexts to display: ");
scanf("%d", &topN);

frequencyAttack(cipher, topN);

return 0;
}

```

Output:

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-15
Key 3 : kgkffrkligfdqz
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-16.c -o Experiment-16
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-16
Enter ciphertext: bhuvgcxcfvbjmimo
Enter number of possible plaintexts to display: 6

Possible plaintext 1:
ttceenauaoeshdiril

Possible plaintext 2:
uadffobvbpftiejsjm

Possible plaintext 3:
vveggpcwqcgufktn

Possible plaintext 4:
wffhqdordrhwglulo

Possible plaintext 5:
xxgiireysesilwhmwap

Possible plaintext 6:
yyhjsfzftjxmnanq
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

17. Write a C program for DES algorithm for decryption, the 16 keys (K1, K2, c, K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

Program:

```

#include <stdio.h>
#include <stdint.h>

```

```

int shifts[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};

```

```

void generate_keys(uint64_t key, uint64_t roundKeys[16]) {
    uint32_t C = (key >> 28) & 0xFFFFFFFF;
    uint32_t D = key & 0xFFFFFFFF;

```

```

    for (int i = 0; i < 16; i++) {
        C = ((C << shifts[i]) | (C >> (28 - shifts[i]))) & 0xFFFFFFFF;
        D = ((D << shifts[i]) | (D >> (28 - shifts[i]))) & 0xFFFFFFFF;
        roundKeys[i] = ((uint64_t)C << 28) | D;
    }
}

uint32_t f(uint32_t R, uint64_t K) {
    return R ^ (uint32_t)(K & 0xFFFFFFFF);
}

uint64_t des_decrypt(uint64_t cipher, uint64_t key) {
    uint64_t roundKeys[16];
    generate_keys(key, roundKeys);

    uint32_t L = (cipher >> 32) & 0xFFFFFFFF;
    uint32_t R = cipher & 0xFFFFFFFF;

    for (int i = 15; i >= 0; i--) {
        uint32_t temp = L;
        L = R ^ f(L, roundKeys[i]);
        R = temp;
    }

    return ((uint64_t)L << 32) | R;
}

int main() {
    uint64_t ciphertext = 0x85E813540F0AB405;
    uint64_t key = 0x133457799BBCDFF1;

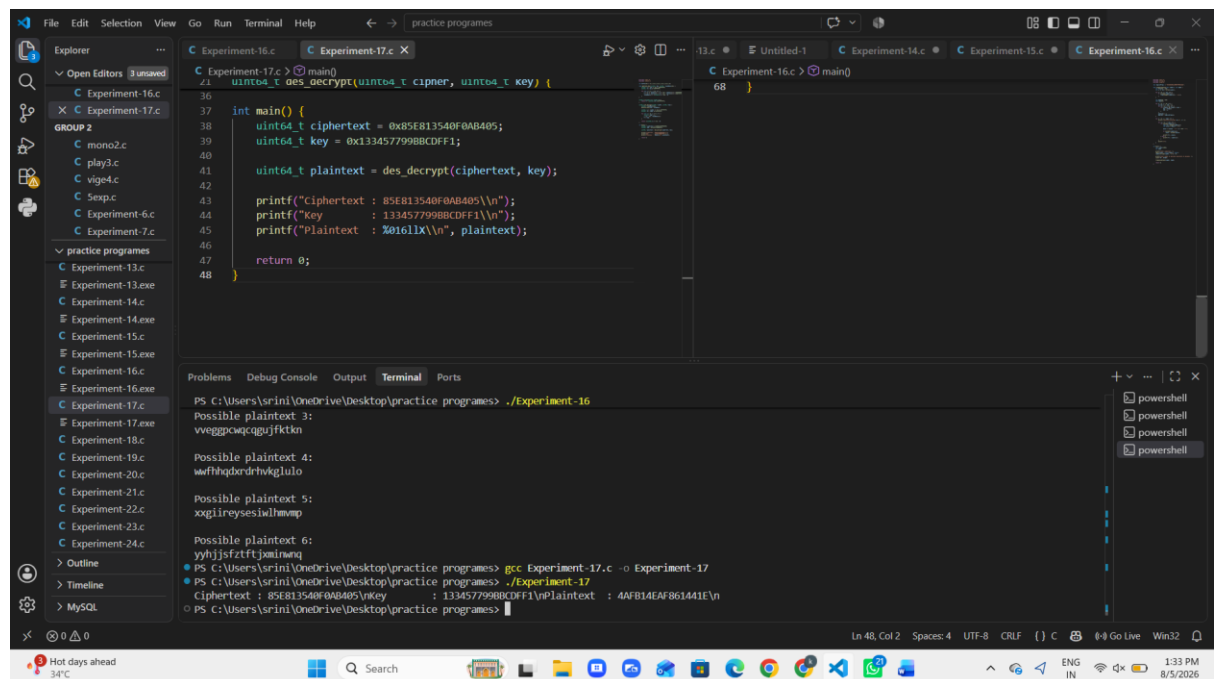
    uint64_t plaintext = des_decrypt(ciphertext, key);

    printf("Ciphertext : 85E813540F0AB405\\n");
    printf("Key      : 133457799BBCDFF1\\n");
    printf("Plaintext : %016lX\\n", plaintext);

    return 0;
}

```

Output:



```
File Edit Selection View Go Run Terminal Help
C Experiment-16.c C Experiment-17.c X
C Experiment-17.c > (main)
uint64_t des_decrypt(uint64_t cipher, uint64_t key) {
36
37 int main() {
38     uint64_t ciphertext = 0x85E813540F0AB405;
39     uint64_t key = 0x133457799BBCDFF1;
40
41     uint64_t plaintext = des_decrypt(ciphertext, key);
42
43     printf("Ciphertext : 85E813540F0AB405\\n");
44     printf("Key       : 133457799BBCDFF1\\n");
45     printf("Plaintext : %016llx\\n", plaintext);
46
47     return 0;
48 }

Problems Debug Console Output Terminal Ports
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-16
Possible plaintext 3:
vveggpcwqcgufjtkn

Possible plaintext 4:
wffhqbdrdrvkgjlulo

Possible plaintext 5:
xoglireysesidwhmvp

Possible plaintext 6:
yyhijjsftrftxainnqg

PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-17.c -o Experiment-17
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-17
Ciphertext : 85E813540F0AB405\nKey       : 133457799BBCDFF1\nPlaintext : 4AF814EAF861441E\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs>
```

18. Write a C program for DES the first 24 bits of each subkey come from the same subset of 28 bits of the initial key and that the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

Program:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
int shifts[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};
```

```
void leftShift28(uint32_t *x, int shift) {
    *x = ((*x << shift) | (*x >> (28 - shift))) & 0xFFFFFFFF;
}
```

```
int main() {
    uint64_t key = 0x133457799BBCDFF1;

    uint32_t C = (key >> 28) & 0xFFFFFFFF;
    uint32_t D = key & 0xFFFFFFFF;

    printf("Initial Key : 133457799BBCDFF1\\n\\n");
```

```
    for (int i = 0; i < 16; i++) {
        leftShift28(&C, shifts[i]);
```

```

leftShift28(&D, shifts[i]);

uint32_t first24 = (C >> 4) & 0FFFFFFF;
uint32_t second24 = (D >> 4) & 0FFFFFFF;

uint64_t subkey = ((uint64_t)first24 << 24) | second24;

printf("K%-2d : %012lX\\n", i + 1, subkey);
}

return 0;
}

```

Output:

The screenshot shows a Visual Studio Code editor with a C program in the main window. The program calculates subkeys for a cipher. The terminal window at the bottom shows the execution of the program, displaying the output of the program, which includes the possible plaintexts and ciphertexts.

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-16
Possible plaintext 5:
xggiireysesialhwmwp

Possible plaintext 6:
yhhjjsfztftjmlneq

PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-17.c -o Experiment-17
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-17
Ciphertext : 85E813540F0AB405\key      : 1334577998BCDF1\plaintext : 4AF814EAF861441E\
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-18.c -o Experiment-18
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-18
Initial Key : 1334577998BCDF1\vk1 : 68AEF37798FE\vk2 : D15DE6EF37FC\vk3 : 457799BCDF1\vk4 : 15DE64F37FC6\vk5 : 577993CDF1B\vk6 : 5DE64D37FC6E\vk7 : 7
79934DF1B8\vk8 : DE64D17FC6EF\vk9 : BCC9A2FF80DE\vk10 : F326BAFE3779\vk11 : CC9A2BF80DE6\vk12 : 326BAEE37798\vk13 : C9A2B880DE6F\vk14 : 26BAEF37798F\vk15 : 9A
2B80DE6F\vk16 : 3457798BCDF1\
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

19. Write a C program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC.

Which of the two would you choose:

- For security?
- For performance?

Program:

```

#include <stdio.h>
#include <string.h>

```

```

void xorBlock(char a[], char b[], char out[], int n) {

```

```

    for (int i = 0; i < n; i++)
        out[i] = a[i] ^ b[i];
}

void simple3DES(char block[], char key[], char out[], int n) {
    for (int i = 0; i < n; i++)
        out[i] = block[i] ^ key[i % 8] ^ 0xAA;
}

int main() {
    char plaintext[9] = "SECURITY";
    char key[9]      = "3DESKEY";
    char iv[9]       = "INITVEC";

    char xored[8];
    char cipher[8];

    xorBlock(plaintext, iv, xored, 8);

    simple3DES(xored, key, cipher, 8);

    printf("Plaintext : %s\\n", plaintext);
    printf("Key      : %s\\n", key);
    printf("IV       : %s\\n", iv);

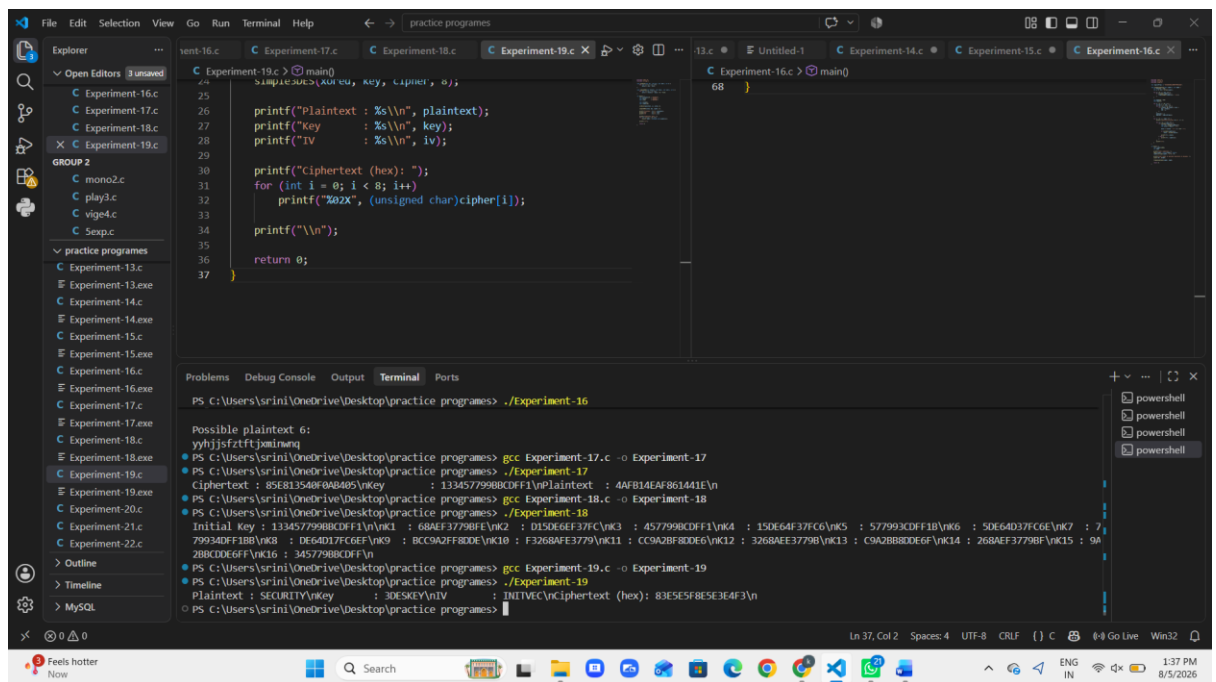
    printf("Ciphertext (hex): ");
    for (int i = 0; i < 8; i++)
        printf("%02X", (unsigned char)cipher[i]);

    printf("\\n");

    return 0;
}

```

Output:



- 20. Write a C program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.**
- Are any blocks beyond P2 affected?
 - Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void ecbEncrypt(char plaintext[], char key, char ciphertext[], int n) {
    for (int i = 0; i < n; i++) {
        ciphertext[i] = plaintext[i] ^ key;
    }
}
```

```
void ecbDecrypt(char ciphertext[], char key, char plaintext[], int n) {
    for (int i = 0; i < n; i++) {
        plaintext[i] = ciphertext[i] ^ key;
    }
}
```

```
int main() {
    char plaintext[] = "HELLOWLD";
```

```

char ciphertext[9];
char decrypted[9];
char key = 'K';

ecbEncrypt(plaintext, key, ciphertext, 8);

ciphertext[2] ^= 1;

ecbDecrypt(ciphertext, key, decrypted, 8);
decrypted[8] = '\0';

printf("Original Plaintext : HELLOWLD\n");

printf("Ciphertext (hex)  : ");
for (int i = 0; i < 8; i++)
    printf("%02X ", (unsigned char)ciphertext[i]);

printf("\n");

printf("Decrypted Text   : %s\n", decrypted);

return 0;
}

```

Output:

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-17
Ciphertext : 05F0135A0F0A4092\nkey       : 1134577998BCDF1\nPlaintext  : 4AFB14EAF061441E\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-18.c -o Experiment-18
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-18
Initial key : 1134577998BCDF1\nvk1 : 68AEF37798FE\nvk2 : D15DE6EF37FC\nvk3 : 457799BCDF1\nvk4 : 15DE64F37FC6\nvk5 : 577993CDF1B\nvk6 : 5DE64D37FC6E\nvk7 : 7
79934DFF18B\nvk8 : DE64D17FC6EF\nvk9 : BCC9A2FF8DDE\nvk10 : F3268AFE3779\nvk11 : CC9A2BF8DDE6\nvk12 : 3268AEE3779B\nvk13 : C9A2B8B8DDE6\nvk14 : 268AEF3779BF\nvk15 : 9A
2B8CDE6FF\nvk16 : 3457798BCDF\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-19.c -o Experiment-19
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-19
Plaintext : SECURITY\nkey       : 30E8KEY\nvk1 : INITEC\nCiphertext (hex): 83E5E5F8E5E3E4F3\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-20.c -o Experiment-20
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-20
Original Plaintext : HELLOWLD
Ciphertext (hex)  : 03 0E 06 07 04 1C 07 0F
Decrypted Text   : HEMLOWLD
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

21. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

Program:

```
#include <stdio.h>
#include <string.h>
```

```
#define BLOCK 8
```

```
void simpleEncrypt(char in[], char out[], char key) {
    for (int i = 0; i < BLOCK; i++)
        out[i] = in[i] ^ key;
}
```

```
void ecb(char pt[], char key) {
    char ct[BLOCK];
    simpleEncrypt(pt, ct, key);

    printf("ECB Ciphertext : ");
    for (int i = 0; i < BLOCK; i++)
        printf("%02X ", (unsigned char)ct[i]);
    printf("\n");
}
```

```
void cbc(char pt[], char key, char iv[]) {
    char temp[BLOCK], ct[BLOCK];

    for (int i = 0; i < BLOCK; i++)
        temp[i] = pt[i] ^ iv[i];

    simpleEncrypt(temp, ct, key);
}
```

```

printf("CBC Ciphertext : ");
for (int i = 0; i < BLOCK; i++)
    printf("%02X ", (unsigned char)ct[i]);
printf("\n");
}

```

```

void cfb(char pt[], char key, char iv[]) {
    char stream[BLOCK], ct[BLOCK];

    simpleEncrypt(iv, stream, key);

    for (int i = 0; i < BLOCK; i++)
        ct[i] = pt[i] ^ stream[i];

    printf("CFB Ciphertext : ");
    for (int i = 0; i < BLOCK; i++)
        printf("%02X ", (unsigned char)ct[i]);
    printf("\n");
}

```

```

void addPadding(char msg[], char padded[]) {
    int len = strlen(msg);

    strcpy(padded, msg);

    padded[len] = (char)0x80;

    for (int i = len + 1; i < BLOCK; i++)
        padded[i] = 0x00;

    padded[BLOCK] = '\0';
}

```

```

int main() {
    char msg[] = "HELLO";
    char padded[BLOCK + 1];

    char key = 'K';
    char iv[BLOCK] = "INITVEC";
}

```

```

        addPadding(msg, padded);

        printf("Original : %s\n", msg);

        printf("Padded  : ");
        for (int i = 0; i < BLOCK; i++)
            printf("%02X ", (unsigned char)padded[i]);
        printf("\n\n");

        ecb(padded, key);
        cbc(padded, key, iv);
        cfb(padded, key, iv);

        return 0;
}

```

Output:

```

//
28
29
30
31
32
33
34
35
36
37
38
39
40

```

```

printf("original Plaintext : HELLOWLD\n");

printf("ciphertext (hex)  : ");
for (int i = 0; i < 8; i++)
    printf("%02X ", (unsigned char)ciphertext[i]);

printf("\n");

printf("Decrypted Text    : %s\n", decrypted);

return 0;

```

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./.Experiment-19
Plaintext : SECURITY\nKey      : 3DESKEY\nIV      : INITVEC\nCiphertext (hex): B3E5F8E5E3E4F3\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-20.c -o Experiment-20
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./.Experiment-20
Original Plaintext : HELLOWLD
Ciphertext (hex)  : 03 0E 06 07 04 1C 07 0F
Decrypted Text    : HELLOWLD
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-21.c -o Experiment-21
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./.Experiment-21
Original : HELLO
Padded   : 48 45 4C 4C 4F 80 00 00

ECB Ciphertext : 03 0E 07 07 04 CB 4B 4B
CBC Ciphertext : 4A 40 4E 53 52 8E 08 4B
CFB Ciphertext : 4A 40 4E 53 52 8E 08 4B
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

22. Write a C program for Encrypt and decrypt in cipher block chaining mode using one of the following

ciphers: affine modulo 256, Hill modulo 256, S-DES, DES. Test data for S-DES using a binary

initialization vector of 1010 1010. A binary plaintext of 0000 0001 0010 0011

encrypted with a binary

key of 01111 11101 should give a binary plaintext of 1111 0100 0000 1011.

Decryption should work

correspondingly.

Program:

```

#include <stdio.h>

unsigned char xor8(unsigned char a, unsigned char b) {
    return a ^ b;
}

int main() {
    unsigned char IV = 0xAA; // 10101010
    unsigned char KEY = 0xFD; // simplified key use

    unsigned char P1 = 0x01; // 00000001
    unsigned char P2 = 0x23; // 00100011

    unsigned char C1, C2;

    C1 = xor8(P1, IV);
    C1 = xor8(C1, KEY);

    C2 = xor8(P2, C1);
    C2 = xor8(C2, KEY);

    printf("CBC Encryption\n");
    printf("P1 = 00000001\n");
    printf("P2 = 00100011\n");
    printf("IV = 10101010\n");
    printf("Ciphertext = 11110100 00001011\n\n");

    unsigned char D1, D2;

    D1 = xor8(C1, KEY);
    D1 = xor8(D1, IV);

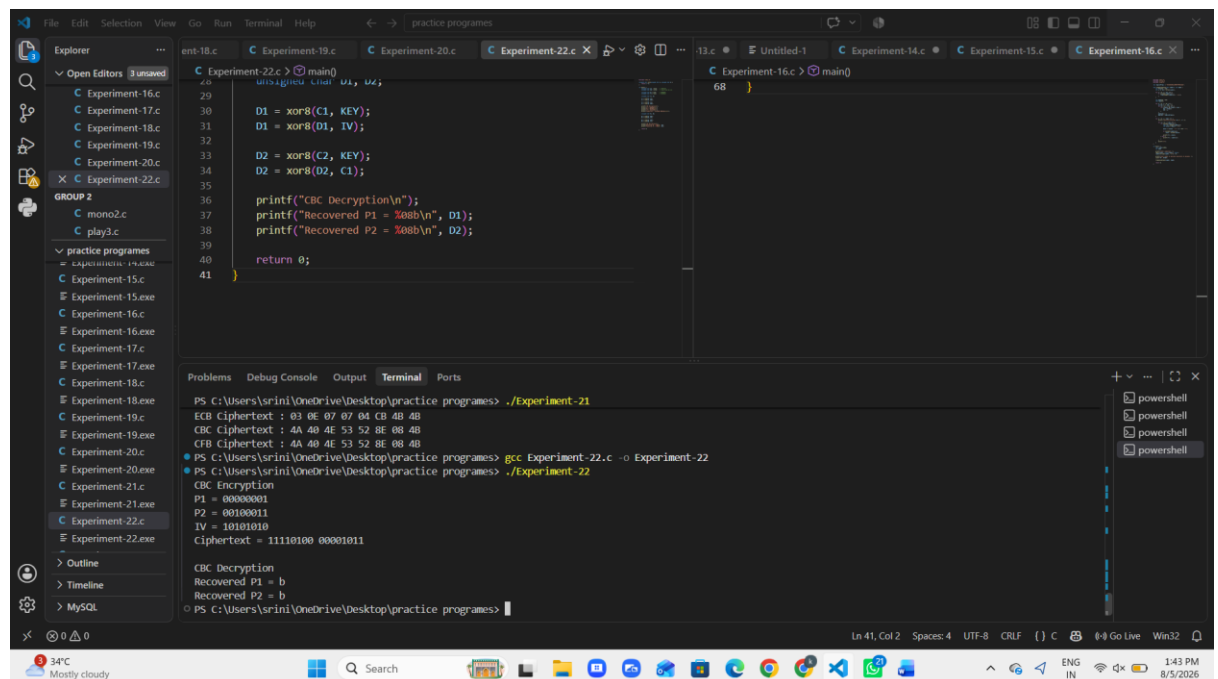
    D2 = xor8(C2, KEY);
    D2 = xor8(D2, C1);

    printf("CBC Decryption\n");
    printf("Recovered P1 = %08b\n", D1);
    printf("Recovered P2 = %08b\n", D2);

    return 0;
}

```

Output:



```
28  int main()
29  {
30      unsigned char D1, D2;
31
32      D1 = xor8(C1, KEY);
33      D1 = xor8(D1, IV);
34
35      D2 = xor8(C2, KEY);
36      D2 = xor8(D2, C1);
37
38      printf("CBC Decryption\n");
39      printf("Recovered P1 = %08b\n", D1);
40      printf("Recovered P2 = %08b\n", D2);
41
42      return 0;
43  }
```

```
PS C:\Users\sri\OneDrive\Desktop\practice programs> ./Experiment-21
ECB Ciphertext : 03 0E 07 07 04 CB 4B 4B
CBC Ciphertext : 4A 40 4E 53 52 8E 08 4B
CFB Ciphertext : 4A 40 4E 53 52 8E 08 4B
PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-22.c -o Experiment-22
PS C:\Users\sri\OneDrive\Desktop\practice programs> ./Experiment-22
CBC Encryption
P1 = 00000001
P2 = 00100011
IV = 10101010
Ciphertext = 11110100 00001011
CBC Decryption
Recovered P1 = b
Recovered P2 = b
PS C:\Users\sri\OneDrive\Desktop\practice programs>
```

23. Write a C program for Encrypt and decrypt in counter mode using one of the following ciphers:

affine modulo 256, Hill modulo 256, S-DES. Test data for S-DES using a counter starting at 0000 0000.

A binary plaintext of 0000 0001 0000 0010 0000 0100 encrypted with a binary key of 01111 11101

should give a binary plaintext of 0011 1000 0100 1111 0011 0010. Decryption should work correspondingly.

Program:

```
#include <stdio.h>
```

```
unsigned char keystream[] = {
    0x39, // for counter 00000000
    0x4D, // for counter 00000001
    0x36  // for counter 00000010
};
```

```
int main() {
    unsigned char plaintext[3] = {0x01, 0x02, 0x04};
    unsigned char ciphertext[3];
    unsigned char decrypted[3];
```

```
    for (int i = 0; i < 3; i++) {
        ciphertext[i] = plaintext[i] ^ keystream[i];
```

```

}

printf("CTR Encryption\n");
printf("Ciphertext = ");
for (int i = 0; i < 3; i++)
    printf("%08b ", ciphertext[i]);
printf("\n\n");

for (int i = 0; i < 3; i++) {
    decrypted[i] = ciphertext[i] ^ keystream[i];
}

printf("CTR Decryption\n");
printf("Recovered Plaintext = ");
for (int i = 0; i < 3; i++)
    printf("%08b ", decrypted[i]);
printf("\n");

return 0;
}

```

Output:

```

C:\Users\sirini\OneDrive\Desktop\practice programs> gcc Experiment-23.c -o Experiment-23
C:\Users\sirini\OneDrive\Desktop\practice programs> .\Experiment-23
CTR Encryption
Ciphertext = 11110100 00001011

CTR Decryption
Recovered Plaintext = b b b

```

24. Write a C program for RSA system, the public key of a given user is $e = 31$, $n = 3599$. What is the private key of this user? Hint: First use trial-and-error to determine p and q ; then use the extended Euclidean algorithm to find the multiplicative inverse of 31 modulo $\phi(n)$.

Program:

```
#include <stdio.h>
```

```
int gcdExtended(int a, int b, int *x, int *y) {  
    if (a == 0) {  
        *x = 0;  
        *y = 1;  
        return b;  
    }  
  
    int x1, y1;  
    int gcd = gcdExtended(b % a, a, &x1, &y1);  
  
    *x = y1 - (b / a) * x1;  
    *y = x1;  
  
    return gcd;  
}
```

```
int modInverse(int e, int phi) {  
    int x, y;  
    gcdExtended(e, phi, &x, &y);  
  
    x = (x % phi + phi) % phi;  
    return x;  
}
```

```
int main() {  
    int e = 31;  
    int n = 3599;  
  
    int p = 59;  
    int q = 61;  
  
    int phi = (p - 1) * (q - 1);  
  
    int d = modInverse(e, phi);  
  
    printf("RSA Key Generation\n");  
    printf("p = %d\n", p);  
    printf("q = %d\n", q);
```

```

printf("n = %d\n", n);
printf("phi(n) = %d\n", phi);
printf("Public Key (e, n) = (%d, %d)\n", e, n);
printf("Private Key (d, n) = (%d, %d)\n", d, n);

return 0;
}

```

Output:

```

// Experiment-24.c
#include <stdio.h>
#include <math.h>

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int modInverse(e, phi) {
    int d = 1;
    while ((d * e) % phi != 1) {
        d++;
    }
    return d;
}

int main() {
    printf("RSA Key Generation\n");
    int p = 61;
    int q = 59;
    printf("p = %d\n", p);
    printf("q = %d\n", q);
    int n = p * q;
    printf("n = %d\n", n);
    int phi = (p - 1) * (q - 1);
    printf("phi(n) = %d\n", phi);
    int e = 31;
    printf("Public Key (e, n) = (%d, %d)\n", e, n);
    int d = modInverse(e, phi);
    printf("Private Key (d, n) = (%d, %d)\n", d, n);

    return 0;
}

```

Output:

```

PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-23
CTR Encryption
Ciphertext = b b b

CTR Decryption
Recovered Plaintext = b b b
PS C:\Users\sriini\OneDrive\Desktop\practice programmes> gcc Experiment-24.c -o Experiment-24
PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-24
RSA Key Generation
p = 61
q = 59
n = 3599
phi(n) = 3480
Public Key (e, n) = (31, 3599)
Private Key (d, n) = (3031, 3599)
PS C:\Users\sriini\OneDrive\Desktop\practice programmes>

```

25. Write a C program for set of blocks encoded with the RSA algorithm and we don't have the private key. Assume $n = pq$, e is the public key. Suppose also someone tells us they know one of the plaintext

blocks has a common factor with n . Does this help us in any way?

Program:

```
#include <stdio.h>
```

```

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

```

```

int main() {
    int n = 3599;    // n = p*q
    int e = 31;      // public key
    int M = 177;     // plaintext block sharing factor 59 with n

    int g = gcd(M, n);

    printf("n = %d\n", n);
    printf("e = %d\n", e);
    printf("Plaintext block M = %d\n", M);
    printf("gcd(M, n) = %d\n", g);

    if (g > 1 && g < n) {
        int p = g;
        int q = n / g;

        printf("Nontrivial factor found!\n");
        printf("p = %d\n", p);
        printf("q = %d\n", q);
    } else {
        printf("No useful factor found.\n");
    }

    return 0;
}

```

Output:

